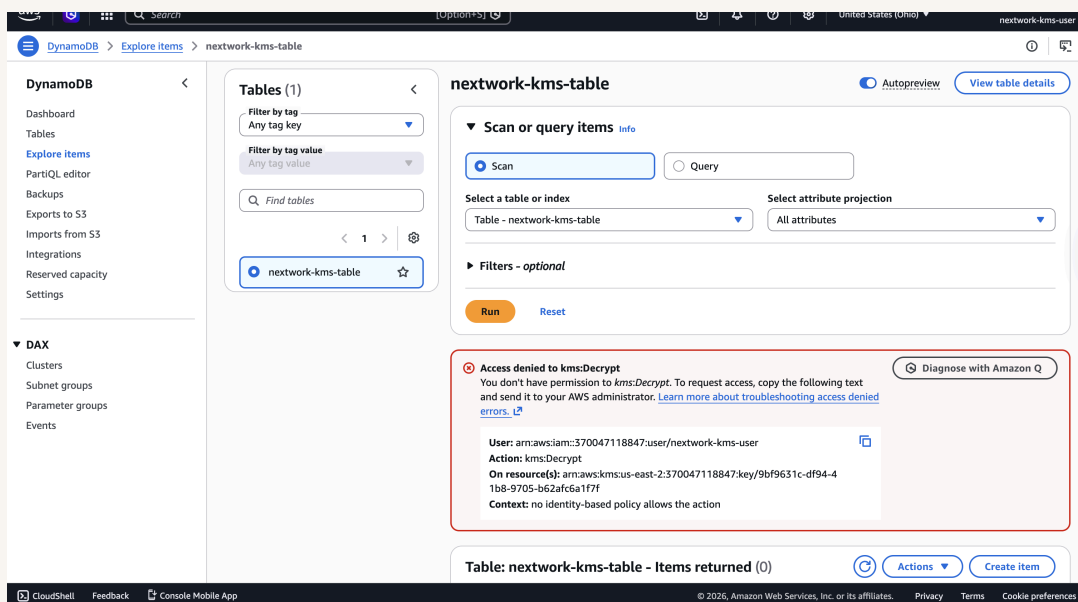




Encrypt Data with AWS KMS



Adeem Akhtar





Introducing Today's Project!

In this project, I will demonstrate the following: 1. Create encryption keys with AWS KMS (Key Management Service). 2. Encrypt a DynamoDB database with a KMS key. 3. Add and retrieve data from your database to test your encryption. 4. Observe how AWS stops unauthorised access to your data. 5. Give a user encryption access. The goal is to secure data in DynamoDB in the cloud environment by encrypting it to prevent unauthorised access.

Tools and concepts

Services I used include KMS, IAM, DynamoDB. Key concepts I learnt include encryption, decryption, policy allocation and policy upgrading.

Project reflection

This project took me approximately 45 minutes. The most challenging part was policy upgrading to the test user. It was most rewarding to test the access of test user.

I chose to do this project today because I am learning AWS security. Something that would make learning with NextWork even better is engaging projects.

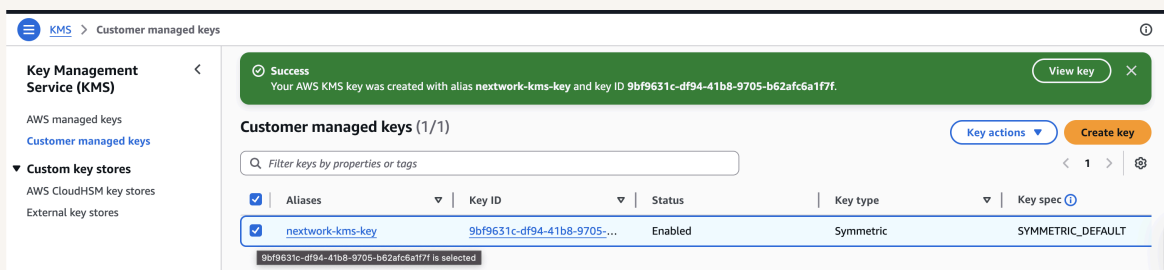


Encryption and KMS

Encryption is the process of transforming data from its original form to a different form called ciphertext. Companies and developers do this to secure data while in transfer or at rest. Encryption keys are used to tell the algorithm how to transform the text from one format to another.

AWS KMS is a service provided by AWS as a security feature. Key management systems are important because it is used to create, manage and use encryption keys that protect the data in the AWS resources.

Encryption keys are broadly categorised as key pairs one for encryption and the other for decryption. I set up a symmetric key because it requires only one key to encrypt as well as to decrypt.





Encrypting Data

My encryption key will safeguard data in DynamoDB, which is a NoSQL database service provided by AWS.

The different encryption options in DynamoDB include the following: 1. AWS owned key (The key is owned and managed by DynamoDB. You are not charged additional fees for using this key.) 2. AWS managed key (The key is stored in your account and managed by AWS Key Management Service (KMS). AWS KMS charges apply.) 3. Customer managed key (The key is stored in your account and managed by you. AWS KMS charges apply.) I selected, [3. Customer managed key].

The screenshot shows the AWS Management Console for the 'nextwork-kms-table' table. At the top, there is a navigation breadcrumb: 'DynamoDB > Tables > nextwork-kms-table > Manage encryption'. Below this is a blue feedback banner that says 'Share your feedback on Amazon DynamoDB' with a 'Share feedback' button and a close icon. The main heading is 'Manage encryption' with an 'Info' link. Below the heading, a note states: 'All data stored in Amazon DynamoDB is fully encrypted at rest. By default, DynamoDB manages the encryption key, and you are not charged any fee for using it.' The 'Encryption at rest' section contains three radio button options for 'Encryption key management': 'AWS owned key' (unselected), 'AWS managed key' (unselected), and 'Customer managed key' (selected). Each option has a brief description. Below the 'Customer managed key' option, there is a text input field containing the ARN: 'arn:aws:kms:us-east-2:370047118847:key/9bf9631c-df94-41b8-9705-b62afc6a1f7'. At the bottom right of the console, there are 'Cancel' and 'Save changes' buttons.



Data Visibility

Rather than controlling who has access to the key, KMS manages user permissions by assigning the IAM policy.

Despite encrypting my DynamoDB table, I could still see the table's items because DynamoDB is designed to decrypt the data on your behalf. When data is requested by an authorised user (like you) or an authorised application. DynamoDB uses transparent data encryption, which means "DynamoDB retrieves the encrypted data, decrypts it with the key, then shows you the decrypted format so you can use it instantly. "

nextwork-kms-table

AutopreviewView table details

▼ Scan or query items Info

☒ Scan

☐ Query

Select a table or index

Table - nextwork-kms-table

Select attribute projection

All attributes

► Filters - optional

Run

Reset

✔ Completed · Items returned: 1 · Items scanned: 1 · Efficiency: 100% · RCUs consumed: 2

Table: nextwork-kms-table - Items returned (1)

Scan started on May 08, 2026, 13:42:15

Actions

Create item

< 1 >

⚙

☐ id (String)

☐ 1

© 2026, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

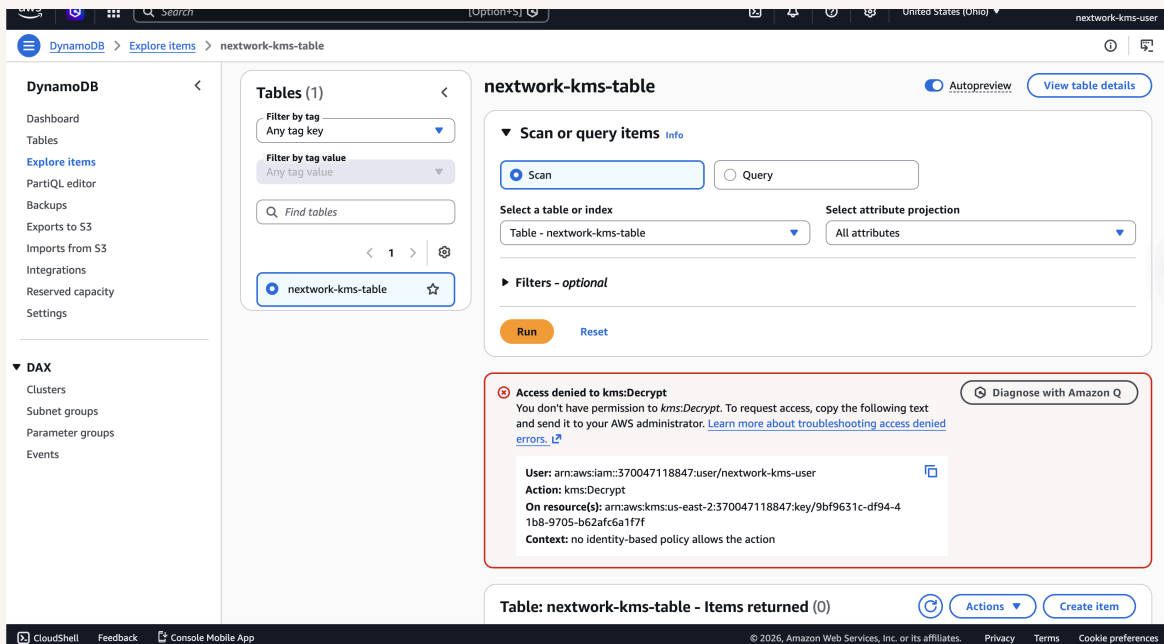
Cookie preferences



Denying Access

I configured a new IAM user to test my data in the DynamoDB. The permission policies I granted this user are "AmazonDynamoDBFullAccess" but not "KMS".

After accessing the DynamoDB table as the test user, I encountered "Access denied to kms:Decrypt" because, although the test user has full access to DynamoDB but does not have access to the private KMS. This confirmed data in DynamoDB has been encrypted with the private key.



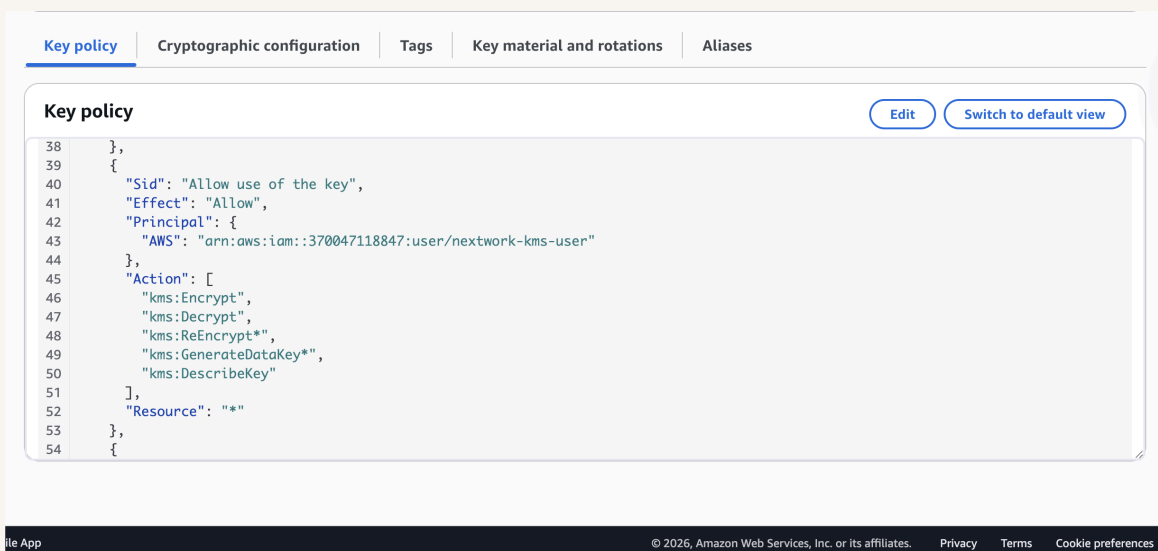


EXTRA: Granting Access

To let my test user use the encryption key, I added the test user as "key user" in my KMS key policy. My key's policy was updated to let the test user encrypt, decrypt, ReEncrypt, GenerateDataKey and DescribeKey.

Using the test user, I retried accessing my table from the test user account. I observed there is a table available with the same name as I created in my admin account, which confirmed that the test user can access the decrypted data using the KMS and the policy has been assigned to the test user successfully.

Encryption secures data instead of storage, devices, or networks. Use access controls (passwords, permissions, MFA) to decide who can access systems or data. I could combine encryption with access controls as access control prevents misuse, encryption limits damage if defenses fail.





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

